Pruebas Unitarias de Guardián Escolar

Johan Smith Santamaria Fernández

Sharik Dayanna Rojas Ibarra

Juan Pablo Chala Ramírez

Tecnólogo en Análisis y Desarrollo de Software

Ficha: 3145556

SENA — Sede La Industria, La Empresa y los Servicios

Neiva, Huila, Colombia

Octubre de 2026

Pruebas Unitarias de Guardián Escolar

1. Introducción

1.1 Propósito y alcance

El presente documento define y documenta las **pruebas unitarias** del proyecto School Guardian, una plataforma de transporte escolar compuesta por una arquitectura de microservicios para el backend y una aplicación web (frontend) desarrollada en Angular.

El alcance de este documento se limita exclusivamente a **pruebas unitarias**: la verificación aislada de la lógica de negocio, validaciones, cálculos, reglas de dominio, mapeos, servicios de aplicación, consultas y componentes de interfaz, ejecutada sin acceso a base de datos, red ni sistema de archivos. Quedan fuera del alcance las pruebas de integración, las pruebas de extremo a extremo (E2E) y las pruebas de rendimiento, que se documentarán en entregables independientes.

Tabla 1

Alcance técnico (basado en el código real del proyecto)

Componente	Stack	Proyectos/servicios	Proyectos de prueba
Backend — microservicios .NET	C# / .NET 10 (`net10.0`), **NUnit**	`ms-configuration`, `ms-exceptional`, `ms-fleet`, `ms-route`, `ms-school-management`, `ms-user-management`, `sg-ms-forgot-information`	Sí, uno por servicio en `tests/{servicio}.Tests`
Backend — IAM	Java 21, Spring Boot 4.1.1, Maven, JUnit 5 + Mockito	`ms-iam`	Sí (`src/test/java`)
Backend — notificaciones	C# / .NET 10, **NUnit**	`ms-notification`	**No existe** (auditoría pendiente)
Gateway	Kong (configuración declarativa `kong.yml`)	`ms-api-gateway`	No aplica (sin código)
Infraestructura de mensajería	Kafka (Docker Compose, tópicos)	`kafka`	No aplica (sin código)

Componente	Stack	Proyectos/servicios	Proyectos de prueba
Frontend	Angular 21, TypeScript 5.9, Vitest 4 + jsdom	Aplicación web `dvlp-web`	Sí (64 archivos `*.spec.ts`)

Módulos funcionales cubiertos (referenciados contra el informe de requisitos):

- **RF-01 Gestión de usuarios**: creación, actualización, activación/desactivación de cuentas por rol y vinculación de estudiantes a acudientes (`ms-user-management`, `ms-school-management`).
- **RF-02 Autenticación**: inicio y cierre de sesión, emisión y renovación de tokens, recuperación de credenciales (`ms-iam` y flujos de `sg-ms-forgot-information`).
- **RF-03 Rutas escolares, flota y asignaciones**: rutas con paradas ordenadas y horarios, buses, conductores, asignaciones (`ms-route`, `ms-fleet`).
- **RF-04 / RF-05 Escaneo QR y notificaciones de bordada**: control de subida y bajada y aviso automático a acudientes (`ms-notification`; integración con el frontend móvil).
- **RF-07 Gestión de viajes**: inicio y finalización del ciclo de viaje (`ms-route`).
- **RF-09 Consulta de rutas**: consultas del acudiente y del conductor.
- **RF-10 Personalización**: tema de colores e idiomas (frontend).

1.2 Audiencia

Tabla 2

Audiencia

Audiencia	Uso esperado
Desarrolladores backend y frontend	Conocer qué probar, convenciones y cómo ejecutar las pruebas
Equipo de QA	Ejecutar, ampliar y mantener la batería de pruebas
Líder técnico	Evaluar cumplimiento de criterios de calidad y cobertura

1.3 Definiciones y acrónimos

Tabla 3

Definiciones y acrónimos

Término	Definición
Prueba unitaria	Prueba que verifica una unidad mínima de comportamiento (método o función) de forma aislada, sin dependencias reales externas.
Unidad	En este proyecto: un caso de uso (`*Service`) de la capa `Application`, una regla de dominio (`*Rules`), una estrategia de búsqueda (`*SearchStrategy`), un filtro de seguridad o un servicio/componente del frontend.
Mock	Objeto que registra las llamadas esperadas y devuelve respuestas programadas, permitiendo verificar interacciones (p. ej. `mock(ProfileJpaRepository.class)` en `ms-iam`).
Stub	Objeto que devuelve datos fijos para que el sistema bajo prueba (SUT) pueda ejecutarse; no verifica interacciones.
Fake	Implementación ligera y funcional de una dependencia (p. ej. `InMemoryBusRepository`) que se usa como sustituto real en la prueba.
Fixture	Conjunto de datos o estado inicial preparado para una prueba.
Cobertura	Porcentaje de código (líneas, ramas o condiciones) ejecutado por la batería de pruebas.
SUT	*System Under Test*: la unidad que se está probando.
AAA	Arrange-Act-Assert: patrón de estructuración de una prueba (preparar, actuar, verificar).
PE / AVL	Partición de equivalencia / Análisis de valores límite: técnicas de diseño de casos de prueba.
TDD	*Test-Driven Development*: desarrollo guiado por pruebas.

1.4 Referencias

Tabla 4

Referencias

Referencia	Documento/artefacto
Requisitos	Informe de Especificación de Requisitos de Software — documento 01 del repositorio de documentación (grupos de requisitos RF-01 a RF-10).
Diseño de software	Documento 04 — Diseño de Software (arquitectura, interfaces gráficas, modelo de datos).

Referencia	Documento/artefacto
Artefactos de código	Proyectos `.csproj`/`pom.xml`/`package.json`, clases de la capa `Application`/`Domain`/`Infrastructure` de cada microservicio, esquema de datos (`db-school-guardian.dbml` ↔ DDL), fuentes del frontend y sus especificaciones `.spec.ts`.

1.5 Control de versiones

Tabla 5

Control de versiones

Versión	Fecha	Descripción de cambios	Autor
1.0	2026-10-10	Versión inicial: definición de la estrategia y del diseño de casos de prueba unitaria basado en el código real del proyecto.	Equipo ADSO

2. Estrategia de pruebas unitarias

2.1 Objetivos

1. Verificar la **lógica de negocio** de los casos de uso sin acceso a base de datos, red ni sistema de archivos.
2. Garantizar que las **validaciones** (unicidad, rangos, formatos, horarios) rechacen datos inválidos y acepten datos válidos.
3. Cubrir **cálculos y reglas de dominio** sensibles: generación y expiración de códigos OTP, ciclo de viaje, control de subida/bajada, políticas de contraseñas y enmascaramiento de datos.
4. Verificar los **servicios de aplicación**, las **búsquedas/configuraciones por tenant** y los **filtros de seguridad** del backend.
5. Verificar en el frontend los **servicios HTTP (mocks de red)**, el **manejo de errores (ProblemDetails)** y el **comportamiento de páginas y componentes**.
6. Servir de **red de regresión** frente a cambios de requisitos y refactorizaciones.

2.2 Qué se prueba

Tabla 6

Qué se prueba

Categoría	Unidades concretas en el proyecto
Lógica de negocio (casos de uso)	`CreateBusService`, `AssignDriverToBusService`, `CreateRouteService`, `StartTripService`, `EndTripService`, `AssignStudentToRouteService`, `CreateStudentService`, `RegisterFamilyService`, `CreateSchoolWithCampusesService`, `ProcessScanService`, `LoginService`, `ForgotPasswordService`, etc.
Validaciones y reglas	`FamilyRules`, `DriverLicenseRules`, `SchoolCampusRequestRules`, `SchoolLocationRequestRules`, `OtpOptions`, validaciones de `PersonRequestDto`, unicidad de placa/identificación/licencia.
Cálculos	Generación de OTP (`OtpCodeGenerator`), expiración de códigos y tokens (`VerificationCodeService`, `JwtTokenProvider`), enmascaramiento (`EmailLogMask`), `SecretHasher`, resúmenes de viajes y consultas del conductor (`GetDriverRouteTodayService`).
Búsquedas y filtros	`NameSearchStrategy`, `EmailSearchStrategy`, `IdentificationSearchStrategy`, `PhoneSearchStrategy`, `PlateSearchStrategy`, `TenantPersonFilter`, `RoleFilteredList`, listas por tenant.
Mapeos	`PersonProfile` (AutoMapper), `StudentScannedEventMapper`, DTOs de bus/vista (`BusViewFieldsTests`).
Seguridad (backend)	`JwtTokenProvider`, `InternalApiKeyFilter`, `JwtAuthenticationFilter`, `GlobalExceptionHandler`, `AuthExceptionHandler`.
Infraestructura de contexto	Pruebas de mapeo del modelo EF (`SettingsContextTests`, `ExceptionalContextTests`) sobre un contexto sin conexión real.
Frontend — servicios	`AuthService`, `BusesService`, `RoutesService`, `StopsService`, `StudentsService`, `ForgotInformationService`, `ThemeContextService`, parsing de `problem-detail`.
Frontend — componentes y páginas	Páginas de administración (buses, rutas, paradas, conductores, familias, acudientes, estudiantes), dashboards, flujos de perfil (cambio de contacto/correo/contraseña) y componentes compartidos.

2.3 Qué no se prueba

Tabla 7

Qué no se prueba

Categoría	Motivo
Código generado	Migraciones EF, esquemas Angular generados por schematics, código de plantillas.
Getters/setters triviales	Sin lógica; agregan ruido sin valor de verificación.
Configuración	`Program.cs`, `ServiceCollectionExtensions`, `appsettings`, clases `*Config`, `SecurityConfig`, `OpenApiConfig`.
Librerías de terceros	NUnit, EF Core, Spring Security, JWT libs, Angular Material, Leaflet, RxJS.
Kafka broker y tópicos	Infraestructura; su consumo se simula en la unidad consumidora.
Gateway Kong	Configuración declarativa sin código (se valida con `kong check`, no con pruebas unitarias).
Interfaz visual/estilos	Se cubre en las pruebas E2E y de revisión visual, fuera de este documento.

2.4 Enfoque de caja blanca con aislamiento de dependencias

Las pruebas siguen un **enfoque de caja blanca**: se escriben a partir de la estructura interna real de las clases (casos de uso, estrategias, filtros) y se mide la cobertura de sentencias, ramas y condiciones. Para aislar el sistema bajo prueba, **toda dependencia externa se simula**:

- Repositorios EF Core → **fakes en memoria** (`InMemoryBusRepository`, `InMemoryPersonRepository`, `InMemoryConfigurationRepositories`, etc.).
- Reloj del sistema → **proveedores de tiempo controlados** (`ManualTimeProvider` en `sg-ms-forgot-information`, `FixedTimeProvider` en `ms-route`).
- HTTP (API IAM / directorio de identidad) → clientes HTTP falsos (`FakeIdentityDirectoryClient`) y `HttpTestingController` en el frontend.
- SMTP → remitente simulado (`FakeNotificationSenders`).

- Repositorios JPA y `PasswordEncoder` → **mocks de Mockito** en `ms-iam`.
- Dispositivos GPS → `FakeGpsDeviceService`.
- Providencia de contextos EF: pruebas de mapeo con `UseSqlServer("Server=unused;...")` que **no abren conexión**.

2.5 Enfoque de desarrollo: pruebas posteriores (estado actual) con transición a TDD

La batería actual del proyecto fue escrita **después de la implementación** (pruebas de caracterización de código existente). Para el frontend se observan además especificaciones de tipo *smoke* (p. ej. `should create`) junto a pruebas de comportamiento reales (servicios HTTP). A partir de este documento:

- Las pruebas existentes se **amplían y completan** (módulos sin cubrir: `ms-notification`, casos de uso de `ms-iam`, viajes y paradas de `ms-route`).
- Para **toda funcionalidad nueva** se adopta **TDD** (rojo → verde → refactorizar), con la prueba fallando primero.
- **Framework de las pruebas .NET: NUnit.** Todas las pruebas de los microservicios .NET se implementan con NUnit (paquete `NUnit` + `NUnit3TestAdapter`); los comandos de ejecución y cobertura se mantienen (ver secciones 3.1 y 9).

2.6 Criterios de entrada

- El código del servicio compila.
- Las dependencias del SUT se pueden reemplazar (inyección por constructor o interfaces de puerto).
- Existen fakes/fixtures para repositorios, reloj, HTTP y mensajería.
- No se requiere base de datos, red ni sistema de archivos para ejecutar (ver sección 9).

2.7 Criterios de salida

- **100 % de las pruebas aprobadas** y **0 pruebas inestables** (flaky) conocidas.
- **Cobertura mínima del 80 %** de líneas y ramas en la capa `Application`/`Domain` de cada microservicio y en los servicios del frontend.
- Toda rama de validación crítica (unicidad, límites, excepciones) probada al menos una vez.
- Las pruebas no dependen del orden de ejecución ni del entorno.

3. Herramientas y entorno

3.1 Backend — microservicios .NET

Tabla 8

microservicios .NET

Herramienta	**Detalle (verificado en los `.csproj` reales)**
Lenguaje / target	C# sobre **.NET 10** (`<TargetFramework>net10.0</TargetFramework>`)
Framework de pruebas	**NUnit** (paquete `NUnit` 4.x) + adaptador **`NUnit3TestAdapter`**
Test SDK	`Microsoft.NET.Test.Sdk` 17.12.0 (ms-forgot-information), 17.14.1 (ms-configuration, ms-exceptional, ms-school-management) y 18.0.0 (ms-fleet, ms-route, ms-user-management)
Mocking	**Sin librerías de mocking** (no se usa Moq/NSubstitute/FakeItEasy): el proyecto emplea **fakes escritos a mano** (`InMemory*Repository`, `Fake*Service`, `ManualTimeProvider`/`FixedTimeProvider`)
Cobertura	`coverlet.collector` 6.0.4 en ms-configuration, ms-exceptional y ms-school-management; en los demás proyectos se usa el collector integrado de code coverage del Test SDK. El collector funciona igual con NUnit (ver sección 9)
IDE	Visual Studio 2022+, JetBrains Rider o VS Code con C# Dev Kit
CLI	`dotnet` (SDK 10)

3.2 Backend — microservicio IAM (ms-iam)

Tabla 9

microservicio IAM (ms-iam)

Herramienta	Detalle (verificado en `pom.xml` real)
Lenguaje / versión	**Java 21** (`<java.version>21</java.version>`)
Framework	**Spring Boot 4.1.1** (parent `spring-boot-starter-parent`)
Framework de pruebas	**JUnit 5 (Jupiter)** mediante `spring-boot-starter-webmvc-test`, `spring-boot-starter-security-test`, `spring-boot-starter-data-jpa-test` y `spring-boot-starter-validation-test`
Mocking	**Mockito** (imports reales en: `InternalProfileServiceTest` con `mock(ProfileJpaRepository.class)` y `mock(PasswordEncoder.class)`)
Cobertura	**No configurada** (no hay plugin JaCoCo en el `pom.xml`); se recomienda su incorporación (ver sección 12)
Build	Maven (presente el wrapper `mvnw` / `mvnw.cmd`)
IDE	IntelliJ IDEA, Eclipse/STS o VS Code con extensiones Java

3.3 Frontend — aplicación web

Tabla 10

aplicación web

Herramienta	Detalle (verificado en `package.json` y `angular.json` reales)
Framework	**Angular 21.2.x** (`@angular/core` ^21.2.9), Angular Material 21, SSR (`@angular/ssr`), `@ngx-translate`
Lenguaje	**TypeScript 5.9.2**
Ejecutor de pruebas	**Vitest 4.0.8** + **jsdom 28** a través del builder `@ngx-env/builder:unit-test` (target `test` de `angular.json`; comando `ng test`)
Mocking HTTP	`@angular/common/http/testing` (`HttpTestingController`)
Gestor de paquetes	npm 10.9.1 (`packageManager: "npm@10.9.1"`)
IDE	VS Code (volor Angular Language Server) o WebStorm

3.4 Configuración del entorno

Tabla 11

Configuración del entorno

Paso	Comando
…NET	`dotnet restore` en cada solución/servicio
Java	`./mvnw -q dependency:resolve` en `ms-iam`
Frontend	`npm ci` en el directorio raíz de la aplicación

Requisitos de máquina: SDK .NET 10, JDK 21, Node.js 20 LTS+, Docker (para la ejecución contenerizada de la sección 9).

4. Estructura y convenciones

4.1 Ubicación de las pruebas en el repositorio

Tabla 12

Ubicación de las pruebas en el repositorio

Capa	Ubicación
Backend .NET	`{servicio}/tests/{servicio}.Tests/`, con subcarpetas que **espejan** la estructura de `src/{servicio}.Api/`: `Application/` (pruebas de casos de uso), `Infrastructure/` (pruebas de contexto de persistencia), `Fakes/` (dobles compartidos: repositorios en memoria, servicios y proveedores de tiempo falsos), `SmokeTests.cs`
Backend Java	`ms-iam/src/test/java/...` con el **mismo paquete** que el código bajo prueba (`com.school_guardian.ms_iam.application.usecase`, `...infrastructure.web.filter`, `...infrastructure.config`)
Frontend	Archivos `*.spec.ts` **colocados junto a** la unidad: `src/app/core/services/auth.service.spec.ts`, `src/app/features/admin/routes-buses/pages/buses/buses.spec.ts`, etc.

4.2 Relación entre clases de código y clases de prueba

Regla **1 a N**: una clase de código puede tener una o varias clases de prueba, pero una clase de prueba siempre se enfoca en **una única unidad** (no se mezclan casos de uso en la

misma clase de prueba, salvo las suites de humo). Las suites por servicio van en `Fakes/` y se comparten solo dentro del proyecto de pruebas.

4.3 Convención de nombres

Tabla 13

Convención de nombres

Capa	Convención	Ejemplo real
.NET	`Metodo_Condicion_ResultadoEsperado`	`ExecuteAsync_WithDuplicatePlate_ShouldThrow` (ms-fleet)
Java	Método descriptivo en minúscula camello	`lookupNormalizesEmailAndReturnsContract` (ms-iam)
Frontend	`describe('unidad', ...)` + `it('comportamiento...')` en narrativa	`describe('AuthService session')` / `it('guarda profileId/personId/email desde la respuesta de login')`

4.4 Patrón de estructuración

AAA (Arrange-Act-Assert), equivalente a Given-When-Then (sintaxis **NUnit**):

```
// Arrange
var repo = new InMemoryBusRepository();
var gpsService = new FakeGpsDeviceService();
var useCase = new CreateBusService(repo, gpsService);

// Act
var busId = await useCase.ExecuteAsync(
    Guid.NewGuid(), DateTime.UtcNow.AddYears(1), Guid.NewGuid(),
    40, "ABC123", 1);

// Assert
Assert.That(busId, Is.Not.EqualTo(Guid.Empty));
```

En el frontend: `beforeEach` (Arrange: `TestBed.configureTestingModule` + fakes de HTTP) → llamada/expectativa con `httpMock.expectOne` (Act) → `expect(...)` (Assert).

4.5 Reglas de calidad de las baterías

- **Independencia**: cada prueba crea sus propios fakes y fixture; no comparte estado entre pruebas.

- **Repetibilidad**: resultados idénticos en cualquier orden; el reloj y los GUID se controlan o no se asumen.
- **Velocidad**: ejecución en milisegundos por prueba (sin E/S real).
- **Sin orden**: no existe dependencia entre pruebas; la paralelización se define explícitamente con `[Parallelizable]` cuando aplica.
- **Una verificación por prueba**: el nombre declara la condición y el resultado esperado.

5. Componentes o unidades a probar

Convenciones de la tabla: la columna *ID* agrupa por módulo; la marca **[sin pruebas]** indica unidades que hoy **no tienen prueba existente** y requieren creación; el resto ya cuenta con pruebas que se deben conservar y ampliar.

5.1 Backend — ms-configuration (RF-02: políticas y ajustes de seguridad)

Tabla 14

ms-configuration (RF-02: políticas y ajustes de seguridad)

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
CON-01	`UpdateSettingService`	`ExecuteAsync`	Actualiza un ajuste de seguridad (intentos de inicio de sesión, periodo de bloqueo) con validación de rango	Alta
CON-02	`UpdatePasswordPolicyService`	`ExecuteAsync`	Actualiza la política de contraseñas (longitud mínima/máxima, requisitos) validando consistencia	Alta
CON-03	`GetSettingService`	`Execute`	Consulta un ajuste por clave/clasificación	Media
CON-04	`ListSettingsService` / `ListPasswordPoliciesService`	`Execute`	Lista de ajustes y políticas	Media

5.2 Backend — ms-fleet (RF-03.6, RF-03.7)

Tabla 15

ms-fleet (RF-03.6, RF-03.7)

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
FLE-01	`CreateBusService`	`ExecuteAsync`	Crea bus; rechaza placa duplicada; genera identificador	Alta
FLE-02	`UpdateBusService` / `ChangeBusStatusService` / `DeleteBusService`	`ExecuteAsync`	Edición, activación/desactivación y eliminación de buses	Alta
FLE-03	`AssignDriverToBusService` / `UnassignDriverFromBusService`	`ExecuteAsync`	Asignación/desasignación de conductor a bus; evita doble asignación	Alta
FLE-04	`GetBusAssignedToDriverService`	`ExecuteAsync`	Obtención del bus asignado a un conductor	Media
FLE-05	`GetBusDetailService` / `ListBusesBasicService`	`ExecuteAsync`	Detalle y listado básico de buses	Media
FLE-06	`SearchBusesService` + `NameSearchStrategy` / `PlateSearchStrategy`	`ExecuteAsync`	Búsqueda por nombre o placa	Media
FLE-07	`ListBrandsService` / `ListModelsService`	`ExecuteAsync`	Catálogos de marcas/modelos	Baja
FLE-08	Proyecciones de vista de bus (`BusViewFieldsTests`)	—	Verificación de los campos expuestos en DTO/vista	Media

Nota: `CreateBusServiceTests.cs` presenta un defecto de estilo (directiva `using Xunit;` duplicada en las líneas 1 y 2); ver sección 12.

5.3 Backend — ms-route (RF-03.1 a RF-03.5, RF-03.8, RF-07.1, RF-09)

Tabla 16

ms-route (RF-03.1 a RF-03.5, RF-03.8, RF-07.1, RF-09)

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
ROU-01	`CreateRouteService`	`ExecuteAsync`	Creación de ruta con datos, paradas ordenadas y horarios; unicidad de ruta	Alta
ROU-02	`UpdateRouteService` / `DeleteRouteService`	`ExecuteAsync`	Actualización y eliminación de rutas	Alta

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
ROU-03	`AssignBusToRouteService`	`ExecuteAsync`	Asignación de un bus a una ruta	Alta
ROU-04	`AssignStudentToRouteService`	`ExecuteAsync`	Asignación de estudiante con parada de subida y bajada	Alta
ROU-05	`StartTripService` **[sin pruebas]** / `EndTripService` **[sin pruebas]**	`ExecuteAsync`	Ciclo de viaje: inicio y finalización (RF-07.1), control de viaje activo	Alta
ROU-06	`GetCurrentTripService` / `GetCurrentRouteService`	`ExecuteAsync`	Consulta del viaje/ruta actual (depende del reloj)	Alta
ROU-07	`GetStudentStopOnRouteService` / `GetStudentRouteService`	`ExecuteAsync`	Punto de subida/bajada y ruta del estudiante	Media
ROU-08	`GetDriverRouteTodayService`	`ExecuteAsync`	Ruta del día para el conductor (RF-09.2)	Media
ROU-09	`SearchRouteService` + `NameSearchStrategy` / `ScheduleSearchStrategy` (Ruta)	`ExecuteAsync`	Búsqueda de rutas por nombre y horario	Media
ROU-10	`SearchStopService` + `NameSearchStrategy` (Parada) **[sin pruebas]**	`ExecuteAsync`	Búsqueda de paradas	Media
ROU-11	`CreateStopService` **[sin pruebas]** / `UpdateStopService` **[sin pruebas]** / `DeleteStopService` **[sin pruebas]** / `AttachStopToRouteService` **[sin pruebas]**	`ExecuteAsync`	CRUD de paradas y anexión a ruta	Media
ROU-12	`ListRouteService` / `ListRouteServiceTenant` / `ListStopService` / `ListCityService`	`ExecuteAsync`	Listados con filtro por tenant/ciudad	Media

5.4 Backend — ms-exceptional

Tabla 17

ms-exceptional

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
EXC-01	`CreateDriverUsageService`	`ExecuteAsync`	Registro de uso excepcional de un conductor con sus validaciones	Alta

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
EXC-02	`CreateRouteUsageService`	`ExecuteAsync`	Registro de uso excepcional de una ruta con sus validaciones	Alta
EXC-03	`ListDriverUsagesService` / `ListRouteUsagesService`	`ExecuteAsync`	Consultas de usos excepcionales	Media

5.5 Backend — ms-school-management

Tabla 18

ms-school-management

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
SCM-01	`CreateSchoolWithCampusesService`	`ExecuteAsync`	Crea institución educativa con sus campus (reglas `SchoolCampusRequestRules`, `SchoolLocationRequestRules`)	Alta
SCM-02	`UpdateSchoolWithCampusesService` / `UpdateSchoolService`	`ExecuteAsync`	Actualización de institución y campus	Alta
SCM-03	`CreateSchoolService` / `DeleteSchoolService`	`ExecuteAsync`	Alta y baja de institución	Alta
SCM-04	`LinkAdminSchoolService` / `GetAdminSchoolService`	`ExecuteAsync`	Vinculación administrador ↔ institución y su consulta	Alta
SCM-05	`SearchSchoolsService` + `NameSearchStrategy`	`ExecuteAsync`	Búsqueda de instituciones por nombre	Media
SCM-06	`ListSchoolsService` / `GetCampusService` / `ListCampusesBySchoolService` / `ListSchoolCampusesService`	`ExecuteAsync`	Listados y consultas de campus	Media

5.6 Backend — ms-user-management (RF-01)

Tabla 19

ms-user-management (RF-01)

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
USM-01	`CreateStudentService` / `CreateDriverService` / `CreateParentService` / `CreateAdminService`	`ExecuteAsync`	Creación de cuentas por rol con validación de identificación única	Alta
USM-02	`RegisterFamilyService`	`ExecuteAsync`	Registro de familia y vinculación de estudiantes (RF-01.4)	Alta
USM-03	`CreateDriverLicenseService` / `UpdateDriverLicenseService` / `DeleteDriverLicenseService`	`ExecuteAsync`	Gestión de licencias de conducción con número único (`DriverLicenseRules`)	Alta
USM-04	`UpdateStudentService` / `UpdateDriverService` / `UpdateParentService` / `UpdateAdminService` / `UpdateFamilyService`	`ExecuteAsync`	Actualización de datos de cuenta (RF-01.2)	Alta
USM-05	`DeleteStudentService` / `DeleteDriverService` / `DeleteParentService` / `DeleteAdminService` / `DeleteFamilyService`	`ExecuteAsync`	Activación/desactivación o eliminación (RF-01.3)	Media
USM-06	`SearchPersonService` + `NameSearchStrategy` / `EmailSearchStrategy` / `IdentificationSearchStrategy` / `PhoneSearchStrategy`	`ExecuteAsync`	Búsqueda de personas por los cuatro criterios	Media
USM-07	`SearchFamilyService` / `GetFamilyService` / `GetFamilyMembersByStudentService` / `ListFamilyService`	`ExecuteAsync`	Familia: consultas y miembros por estudiante	Media
USM-08	`GetStudentService` / `GetDriverService` / `GetParentService` / `GetAdminService` / `GetDriverLicenseService` / `List*Service`	`ExecuteAsync`	Consultas individuales y listados	Media

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
USM-09	`TenantPersonFilter` / `RoleFilteredList`	—	Aislamiento por tenant y por rol (no se filtran datos ajenos)	Alta
USM-10	`PersonProfile` (AutoMapper) / `DriverLicenseView`	—	Proyecciones de persona y licencia hacia DTO/vista	Media

5.7 Backend — sg-ms-forgot-information (RF-02: recuperación de credenciales, cambios de contacto)

Tabla 20

sg-ms-forgot-information (RF-02: recuperación de credenciales, cambios de contacto)

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
FGT-01	`VerificationCodeService`	`GenerateAsync` / `VerifyAsync`	Generación, validación y expiración de códigos de verificación	Alta
FGT-02	`ForgotPasswordService` / `VerifyPasswordResetCodeService` / `ResetPasswordService`	`ExecuteAsync`	Flujo de restablecimiento de contraseña (no revela existencia de cuentas)	Alta
FGT-03	`RequestPhoneChangeService` / `RequestPhoneVerificationService` / `VerifyPhoneChangeIdentityService` / `CheckPhoneVerificationService` / `ResendPhoneVerificationService`	`ExecuteAsync`	Cambio de número de teléfono con verificación por SMS	Alta
FGT-04	`RequestEmailChangeService` / `VerifyEmailChangeCodeService` / `SubmitNewEmailService` / `ConfirmEmailChangeService` / `ResendNewEmailService`	`ExecuteAsync`	Cambio de correo electrónico con verificación en dos pasos	Alta
FGT-05	`OtpCodeGenerator` + `OtpOptions`	`Generate`	Generación de OTP de 6 dígitos (rango 000000–999999)	Alta

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
FGT-06	`SecretHasher` / `EmailLogMask` / `InputValidators`	varios	Hasheo de secretos, enmascaramiento de correos en logs, validadores de entrada	Media
FGT-07	`TransactionalEmails` (plantillas) / `SmtpEmailSender` / `IdentityDirectoryHttpClient`	varios	Plantillas y transporte de correos; cliente HTTP con el directorio de identidad	Media

5.8 Backend — ms-notification (RF-04, RF-05, RF-08) — sin proyecto de pruebas

Tabla 21

sin proyecto de pruebas

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
NOT-01	`ProcessScanService`	`ExecuteAsync`	Procesa el escaneo QR y decide subida/bajada (ON_BOARD/OFF_BOARD) y destinatarios	Alta
NOT-02	`RegisterDeviceService`	`ExecuteAsync`	Registro/desregistro del token de dispositivo del acudiente	Alta
NOT-03	`GetGuardianNotificationsService`	`ExecuteAsync`	Consulta de alertas y notificaciones de un acudiente	Media
NOT-04	`StudentScannedConsumer` / `StudentScannedEventMapper` / `KafkaEventPublisher`	varios	Consumo del evento de escaneo y publicación en el bus; mapeo del evento	Media

5.9 Backend — ms-iam (RF-02, RF-04.1, RF-08)

Tabla 22

ms-iam (RF-02, RF-04.1, RF-08)

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
IAM-01	`LoginService` **[sin pruebas]**	`login`	Autenticación con credenciales, validación de contraseña y emisión de token (RF-02.1)	Alta

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
IAM-02	`RefreshTokenService` **[sin pruebas]** / `LogoutService` **[sin pruebas]**	`refresh` / `logout`	Renovación y revocación de tokens con lista de denegación (RF-02.2)	Alta
IAM-03	`CreateProfileService` **[sin pruebas]**	`execute`	Creación de perfil con rol y evento de dominio	Alta
IAM-04	`InternalProfileService`	`lookup`	Consulta interna de perfil por correo normalizado (usado por otros servicios)	Alta
IAM-05	`GetProfileService` / `UpdateAdminSchoolService` **[sin pruebas]**	`execute`	Consulta de perfil y actualización de institución del administrador	Media
IAM-06	`GetTermsStatusService` **[sin pruebas]** / `RecordTermsAcceptanceService` **[sin pruebas]** / `ListTermsAcceptancesService` **[sin pruebas]**	`execute`	Aceptación y estado de términos y condiciones	Media
IAM-07	`JwtTokenProvider` **[sin pruebas]**	`generate` / `validate`	Generación, firma y validación de tokens JWT, expiración	Alta
IAM-08	`InternalApiKeyFilter`	`doFilterInternal`	Validación de API key para endpoints internos	Alta
IAM-09	`JwtAuthenticationFilter` **[sin pruebas]**	`doFilterInternal`	Autenticación por JWT en peticiones del frontend	Alta
IAM-10	`GlobalExceptionHandler` / `AuthExceptionHandler`	—	Traducción de excepciones a respuestas estructuradas	Media
IAM-11	`PersonCreatedConsumer` **[sin pruebas]** / `KafkaDomainEventPublisher` **[sin pruebas]** / `KafkaConsumerConfig` / `KafkaProducerConfig`	—	Consumo/publicación de eventos de dominio	Media

5.10 Frontend — aplicación web (RF-01 a RF-10 en interfaz)

Tabla 23

aplicación web (RF-01 a RF-10 en interfaz)

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
FE-01	`AuthService`	`login` / `logout`	Gestión de sesión JWT, persistencia de `profileId`/`personId`/correo	Alta

ID	Módulo/Clase	Método/Función	Descripción	Prioridad
FE-02	`StudentsService` / `BusesService` / `RoutesService` / `StopsService`	CRUD HTTP	Consumo de los endpoints de administración con mocks de red	Alta
FE-03	`ForgotInformationService`	flujos de recuperación	Peticiones de recuperación/cambio de contacto (correo, teléfono, contraseña)	Alta
FE-04	`ThemeContextService`	—	Persistencia y aplicación del tema (RF-10.1)	Media
FE-05	`problem-detail` (core/http)	parse	Transformación de respuestas de error ProblemDetails del backend a errores manejables	Alta
FE-06	Páginas admin: `Buses`, `Routes`, `Stops`, `Drivers`, `Families`, `Guardians`, `Students`	—	Comportamiento de pantalla: carga, filtros, formularios, mensajes	Alta
FE-07	Dashboards: `dashboard-admin`, `dashboard-superadmin`	—	Resúmenes y navegación por rol	Media
FE-08	Perfil: `change-contact` (pasos código/teléfono/reset), `change-email`, `change-password`	—	Flujos guiados de cambio de datos de contacto	Alta
FE-09	Feature pública (login/recuperación) y `superadmin` (admins, escuelas)	—	Autenticación pública y administración global	Media
FE-10	Componentes compartidos (`shared/components`, 19 especificaciones)	—	Reutilizables: tablas, diálogos, controles	Media
FE-11	Guards, interceptores y validadores (`core/guards`, `core/interceptors`, `core/validators`)	—	Protección de rutas, adjuntado de token, validaciones de formularios	Alta

6. Diseño de casos de prueba unitaria

6.1 Técnicas aplicadas

Las pruebas de la tabla 6.3 aplican de forma explícita las siguientes técnicas de caja blanca:

- **Partición de equivalencia (PE)**: los datos de entrada se agrupan en clases equivalentes (válidos / inválidos) y se diseña al menos un caso por clase. Ejemplo: identificación `null`, `""`, formato inválido y formato válido.
- **Análisis de valores límite (AVL)**: se prueban los valores en los bordes de cada rango. Ejemplos: longitud mínima/máxima de contraseña, OTP `000000` y `999999`, capacidad de bus `0`, expiración en el instante exacto.
- **Pruebas de excepciones y errores**: se verifica que el SUT lance la excepción correcta (o devuelva el error estructurado) ante estados inválidos (`Assert.That(..., Throws.TypeOf<T>())` en NUnit, `assertThrows` en JUnit, `expect(() => ...).toThrow()` en Vitest).
- **Cobertura de ramas, sentencias y condiciones**: cada `if`/`switch` y cada operador lógico compuesto (AND/OR) del SUT debe ejercitarse al menos una vez en veradero y una en falso; se mide con la herramienta de cobertura de cada capa (ver apartados 3 y 9).

6.2 Formato de los casos

Cada caso se documenta en la tabla con: **ID**, **nombre**, **unidad probada**, **requisito relacionado** (referencias reales RF-xx del informe de requisitos), **descripción y objetivo**, **precondiciones y configuración** (fixtures/mocks), **datos de entrada**, **resultado esperado**, **resultado obtenido**, **estado** y **tipo**.

Estado: al cierre de la redacción, las pruebas **no se han ejecutado** (ver sección 10); el campo *resultado obtenido* queda como «—» y el estado en «No ejecutado». Los casos marcados como **[por crear]** aún no tienen prueba implementada en el código.

6.3 Casos de prueba unitaria

Tabla 24

Casos de prueba unitaria

ID	Caso (nombre)	Unidad probada	Requisito	Descripción y objetivo	Precondiciones y configuración	Datos de entrada	Resultado esperado	Resultado obtenido	Estado	Tipo
PU-001	Crear bus con placa duplicada rechaza la operación	`CreateBusService.ExecuteAsync` (ms-fleet)	RF-03.6	Verificar que una placa ya registrada impide el alta y lanza excepción	`InMemoryBusRepository` pre-cargado con un bus placa `ABC123`; `FakeGpsDeviceService`	Misma placa `ABC123`, capacidad 40, período 1	`ArgumentException`; no se crea un segundo bus	—	No ejecutado	Negativa / Excepción
PU-002	Crear bus con datos válidos devuelve identificador	`CreateBusService.ExecuteAsync` (ms-fleet)	RF-03.6	Verificar el flujo feliz del alta de un bus	`InMemoryBusRepository` vacío; `FakeGpsDeviceService`	Placa `ABC123`, capacidad 40, vencimiento licencia +1 año	`Guid` no vacío; bus persistido en el fake	—	No ejecutado	Positiva
PU-003	Crear bus con capacidad límite 0 o negativa	`CreateBusService.ExecuteAsync` (ms-fleet)	RF-03.6	**AVL**: validar el borde inferior de la capacidad	Fake repositorio y GPS	Capacidad `0` y `-1`	Excepción de argumento (capacidad inválida)	—	No ejecutado	Valor límite
PU-004	Asignar estudiante a parada inexistente	`AssignStudentToRouteService.ExecuteAsync` (ms-route)	RF-03.8	Verificar que no se asigna con paradas no válidas en la ruta	Fakes de repositorios de ruta/parada; ruta con 2 paradas	Parada de subida que no pertenece a la ruta	Excepción controlada; sin asignación creada	—	No ejecutado	Negativa / Excepción

ID	Caso (nombre)	Unidad probada	Requisito	Descripción y objetivo	Precondiciones y configuración	Datos de entrada	Resultado esperado	Resultado obtenido	Estado	Tipo
PU-005	Viaje actual sin viaje activo devuelve vacío	`GetCurrentTripService.ExecuteAsync` (ms-route)	RF-07.1	**AVL/PE**: comportamiento cuando no existe viaje en curso	`FixedTimeProvider` en una hora sin viaje; repositorio de viajes vacío	Consulta `/current` sin viajes	Respuesta vacía/`null` por contrato, sin excepción	—	No ejecutado	Valor límite
PU-006	Iniciar un segundo viaje con viaje activo **[por crear]**	`StartTripService.ExecuteAsync` (ms-route)	RF-07.1	Verificar la regla de exclusividad del viaje activo (rama de condición)	`FixedTimeProvider`; viaje ya iniciado y sin finalizar	`StartTrip` con el mismo bus/ruta	Rechazo o excepción controlada; primer viaje intacto	—	No ejecutado	Negativa / Excepción
PU-007	Finalizar viaje sin viaje activo **[por crear]**	`EndTripService.ExecuteAsync` (ms-route)	RF-07.1	**AVL**: cierre de viaje inexistente no debe corromper estado	Repositorio de viajes sin viajes activos	`EndTrip` sobre un viaje inactivo	No-op o error controlado; estado consistente	—	No ejecutado	Valor límite
PU-008	Horario de ruta rechaza sábado y domingo	Reglas de horario de `CreateRouteService` (ms-route)	RF-03.1	**AVL**: el horario está restringido a lunes–viernes	Fakes de ruta/parada	Días `Saturday` y `Sunday` en el horario	Rechazo del día no hábil	—	No ejecutado	Valor límite
PU-009	Búsqueda de ruta por nombre parcial	`SearchRouteService` + `NameSearchStrategy` (ms-route)	RF-09	**PE**: coincidencia parcial, insensible a mayúsculas/acentos	Fake repositorio con rutas «Colegio A», «Colegio B»	`query = "colegio"`	Devuelve ambas rutas	—	No ejecutado	Positiva

ID	Caso (nombre)	Unidad probada	Requisito	Descripción y objetivo	Precondiciones y configuración	Datos de entrada	Resultado esperado	Resultado obtenido	Estado	Tipo
PU-010	Listado de rutas aislado por tenant	`ListRouteServiceTenant` (ms-route)	RF-03.1	Verificar que el tenant no ve datos de otro tenant	Fakes con rutas de dos tenants	Consulta con `tenantId` A	Solo rutas del tenant A	—	No ejecutado	Positiva (partición)
PU-011	Actualizar ajuste con intentos de sesión fuera de rango	`UpdateSettingService.ExecuteAsync` (ms-configuration)	RF-02	**AVL**: borde inferior y superior de la política de intentos	`InMemoryConfigurationRepositories`	Intentos `0` y `10` (máximo de la política)	Excepción por valor fuera de rango	—	No ejecutado	Valor límite
PU-012	Política con longitud mínima mayor que la máxima	`UpdatePasswordPolicyService.ExecuteAsync` (ms-configuration)	RF-02	**AVL/consistencia**: combinación lógicamente inválida	Fake de repositorio de políticas	Mínima `12`, máxima `6`	Excepción; política sin cambios	—	No ejecutado	Valor límite
PU-013	Crear estudiante con identificación duplicada	`CreateStudentService.ExecuteAsync` (ms-user-management)	RF-01.1	Verificar unicidad de identificación entre personas	`InMemoryPersonRepository` con un estudiante con ID `12345`	Mismo documento `12345`, rol estudiante	Excepción de unicidad	—	No ejecutado	Negativa / Excepción
PU-014	Crear estudiante con identificación nula o vacía	`CreateStudentService.ExecuteAsync` (ms-user-management)	RF-01.1	**PE/AVL**: clases de entrada inválidas (`null`, `""`, espacios)	Fake repositorio vacío	`null`, `""`, `" "`	Validación falla (error de entrada) para las tres clases	—	No ejecutado	Valor límite / Negativa

ID	Caso (nombre)	Unidad probada	Requisito	Descripción y objetivo	Precondiciones y configuración	Datos de entrada	Resultado esperado	Resultado obtenido	Estado	Tipo
PU-015	Registrar licencia con número duplicado	`CreateDriverLicenseService.ExecuteAsync` (ms-user-management)	RF-03.7	Verificar unicidad del número de licencia (`DriverLicenseRules`)	`InMemoryDriverLicenseRepository` con una licencia	Número de licencia repetido	Excepción; licencia no creada	—	No ejecutado	Negativa / Excepción
PU-016	Código de verificación vencidos es rechazado	`VerificationCodeService.VerifyAsync` (sg-ms-forgot-information)	RF-02	**AVL**: expiración del OTP en el borde temporal	`ManualTimeProvider`; código generado con TTL de 5 min	Verificación 6 min después de generar	Rechazo por expiración	—	No ejecutado	Valor límite
PU-017	OTP generado en rango de 6 dígitos	`OtpCodeGenerator.Generate` (sg-ms-forgot-information)	RF-02	**AVL/PE**: límites `000000` y `999999`; formato fijo	`OtpOptions` con longitud 6	1000 generaciones aleatorias	Todos los códigos en `[000000, 999999]` con longitud 6	—	No ejecutado	Valor límite / Positiva
PU-018	Restablecimiento con correo no registrado no revela la cuenta	`ForgotPasswordService.ExecuteAsync` (sg-ms-forgot-information)	RF-02	Seguridad: respuesta genérica idéntica (evita enumeración de cuentas)	`FakeVerificationRequestRepository` vacío; `FakeNotificationSenders`	Correo inexistente	Misma respuesta que un correo existente; sin envío	—	No ejecutado	Negativa
PU-019	Nueva contraseña por debajo de la mínima de la política	`ResetPasswordService.ExecuteAsync` (sg-ms-forgot-information)	RF-02	**AVL**: borde inferior de longitud de contraseña	Política mínima `PwdMinLength = 8`; código válido	Contraseña de 7 caracteres	Rechazo por política	—	No ejecutado	Valor límite

ID	Caso (nombre)	Unidad probada	Requisito	Descripción y objetivo	Precondiciones y configuración	Datos de entrada	Resultado esperado	Resultado obtenido	Estado	Tipo
PU-020	Enmascaramiento de correo en logs	`EmailLogMask` (sg-ms-forgot-information)	RF-02	Verificar que el correo se enmascara parcialmente en registros	Instancia directa de `EmailLogMask`	`juan.perez@example.com`	Máscara que oculta parte del usuario y el dominio	—	No ejecutado	Positiva
PU-021	Escaneo de estudiante sin asignación a ruta activa	`ProcessScanService.ExecuteAsync` (ms-notification) **[por crear]**	RF-04.2 / RF-08.1	Verificar manejo de escaneo de un estudiante sin ruta en curso o sin asignación	Dependencias falsas: repositorios de boarding, alertas, destinatarios; eventos Kafka simulados	Scan QR de estudiante sin asignación	No se genera boarding; alerta/incidente controlado	—	No ejecutado	Negativa / Excepción
PU-022	Subida válida genera evento ON_BOARD y notifica	`ProcessScanService.ExecuteAsync` (ms-notification) **[por crear]**	RF-04.2 / RF-05.1 / RF-05.2	Verificar el flujo feliz: registro de subida, evento y destinatarios	Fakes con estudiante asignado a ruta activa; reloj fijo	Scan QR válido de subida	`Boarding` creado; evento `StudentScanned` publicado; acudientes notificados	—	No ejecutado	Positiva
PU-023	Login con credenciales correctas emite token	`LoginService.login` (ms-iam) **[por crear]**	RF-02.1	Verificar emisión de JWT con claims de rol tras autenticación	Mocks Mockito: repositorio de perfiles, `PasswordEncoder`, `TokenProvider`	Credenciales válidas de un perfil activo	Respuesta con `accessToken`/`refreshToken` y perfil	—	No ejecutado	Positiva

ID	Caso (nombre)	Unidad probada	Requisito	Descripción y objetivo	Precondiciones y configuración	Datos de entrada	Resultado esperado	Resultado obtenido	Estado	Tipo
PU-024	Login con contraseña incorrecta rechaza	`LoginService.login` (ms-iam) **[por crear]**	RF-02.1	Verificar la rama de excepción de credenciales inválidas	Mocks; `PasswordEncoder.matches` devuelve `false`	Correo válido + contraseña incorrecta	`ResponseStatusException` 401; sin token emitido	—	No ejecutado	Negativa / Excepción
PU-025	Renovación con token en lista de denegación	`RefreshTokenService.refresh` (ms-iam) **[por crear]**	RF-02.2	Verificar que tokens revocados no se renuevan	`InMemoryTokenDenyList` con el token; mocks de repositorio	Token revocado + `refreshToken`	Rechazo del refresco	—	No ejecutado	Negativa / Excepción
PU-026	Token vencido se invalida	`JwtTokenProvider.validate` (ms-iam) **[por crear]**	RF-02.2	**AVL**: expiración en el borde temporal del JWT	`JwtTokenProvider` con reloj/`JwtParser` controlado	Token con expiración en el pasado	Token declarado inválido	—	No ejecutado	Valor límite
PU-027	API key interna inválida es rechazada	`InternalApiKeyFilter.doFilterInternal` (ms-iam)	RF-02	Verificar protección de endpoints internos	Mock de `FilterChain` y `HttpServletRequest/Response`	Header `X-Api-Key` incorrecto	Respuesta 401; `FilterChain` no invocado	—	No ejecutado	Negativa
PU-028	Excepción de dominio se traduce a respuesta estructurada	`GlobalExceptionHandler` (ms-iam)	RF-02	Verificar formato `ErrorResponseDto` ante excepción conocida	Instancia del handler; excepción de dominio lanzada	Petición que dispara `ProfileAlreadyExistsException`	Respuesta estructurada con código y mensaje	—	No ejecutado	Positiva

ID	Caso (nombre)	Unidad probada	Requisito	Descripción y objetivo	Precondiciones y configuración	Datos de entrada	Resultado esperado	Resultado obtenido	Estado	Tipo
PU-029	Login del frontend guarda profileId/personId	`AuthService.login` (frontend)	RF-02.1	Verificar persistencia de la sesión en el cliente	`HttpTestingController`; token JWT de prueba generado en el spec	Respuesta de `/login` con `accessToken`, `profileId`, `personId`, correo	Se persisten campos de sesión; suscripción completa	—	No ejecutado	Positiva
PU-030	Error ProblemDetails del backend se maneja	`problem-detail` / `AuthService` (frontend)	RF-02	Verificar el parseo de respuestas de error (negativa)	`HttpTestingController` devolviendo 401/400 con cuerpo ProblemDetails	Respuesta de error con `status`, `title`, `detail`	Error tipado consumible por la UI; sin excepción no controlada	—	No ejecutado	Negativa
PU-031	Payload de error no parseable no rompe el flujo	`problem-detail.parse` (frontend)	RF-02	**AVL/PE**: cuerpo vacío o no JSON	`HttpTestingController` con respuestas 404 sin cuerpo y 500 HTML	Cuerpo `""` o `null`	Objeto por defecto; no lanza	—	No ejecutado	Valor límite
PU-032	`BusesService` mapea la respuesta de listado	`BusesService.list` (frontend)	RF-03.6	Verificar contracto HTTP: query de búsqueda y mapeo a modelo	`HttpTestingController`; mock de respuesta `Bus[]`	Petición GET con `search`	Se envía la query correcta y se mapean los buses	—	No ejecutado	Positiva

Ampliación: los casos PU-001 a PU-032 son la batería inicial. Cada nueva unidad referenciada en la sección 5 debe generar sus propios casos siguiendo la plantilla del anexo A (sección 16), con IDs consecutivos (PU-033, PU-034, …).

7. Manejo de dependencias (mocks, stubs, fakes)

7.1 Dependencias simuladas y cómo se configuran

Tabla 25

Dependencias simuladas y cómo se configuran

Dependencia	Naturaleza real	Dónde se simula	Qué la simula	Cómo se configura
Base de datos EF Core (SQL Server)	Repositorios `*Repository`	Todos los servicios .NET	**Fakes** `InMemory*Repository` (p. ej. `InMemoryBusRepository`, `InMemoryPersonRepository`, `InMemoryExceptionalRepositories`, `InMemoryConfigurationRepositories`)	Se inyecta en el constructor del caso de uso en lugar del repositorio real; persisten en colecciones en memoria
Contexto EF (mapeo)	`SettingsContext`, `ExceptionalContext`	Pruebas `Infrastructure/*ContextTests`	Contexto con `UseSqlServer("Server=unused;...")`	No abre conexión: solo consulta metadatos del modelo
Bases JPA + encoder	`ProfileJpaRepository`, `PasswordEncoder`	`ms-iam`	**Mocks de Mockito** (`mock(...)` + `when(...).thenReturn(...)`)	Mockito (JUnit 5); se programan respuestas y se verifican interacciones
Reloj del sistema	`DateTime.UtcNow` / `Clock`	`ms-route`, `sg-ms-forgot-information`	`FixedTimeProvider`, `ManualTimeProvider`	Proveedor de tiempo controlado inyectado; permite probar expiración y «hoy»
Dispositivo GPS	Servicio GPS externo	`ms-fleet`	`FakeGpsDeviceService`	Devuelve un identificador de dispositivo simulado
API de directorio de identidad (HTTP)	`IdentityDirectoryHttpClient`	`sg-ms-forgot-information`	`FakeIdentityDirectoryClient`	Implementación falsa del cliente HTTP
SMTP / correo	`SmtpEmailSender`, plantillas	`sg-ms-forgot-information`	`FakeNotificationSenders` / remitente simulado	Sustituye el transporte real por envío en memoria
Verificación por SMS	Proveedor de SMS externo	`sg-ms-forgot-information`	`FakeSmsVerificationService`	Devuelve códigos/estados deterministas

Dependencia	Naturaleza real	Dónde se simula	Qué la simula	Cómo se configura
Repositorio de solicitudes de verificación	Repositorio EF	`sg-ms-forgot-information`	`FakeVerificationRequestRepository`	Registro en memoria de solicitudes y códigos
HTTP del frontend	`HttpClient` hacia el gateway	Frontend (servicios)	`HttpTestingController` (`provideHttpClientTesting()`)	`expectOne(...)`, `flush(...)`/`error(...)` por prueba; `afterEach(() => httpMock.verify())`
Mensajería Kafka	Broker	`ms-notification`, `ms-iam`	Publicadores/ consumidores en memoria en los fakes	Se inyectan implementaciones falsas del publicador de eventos

7.2 Criterios para elegir mock, stub o fake

Tabla 26

Criterios para elegir mock, stub o fake

Doble	Cuándo usarlo	Ejemplo en el proyecto
Fake	Cuando la dependencia tiene **lógica propia** que conviene reutilizar (repositorios con estado, búsquedas) o cuando se necesita un contrato funcional completo	`InMemory*Repository`, `ManualTimeProvider`
Stub	Cuando solo se necesita **devolver un valor fijo** para que el SUT avance; no interesa verificar interacciones	`FakeGpsDeviceService`, respuestas preprogramadas en `when(...).thenReturn(...)`
Mock	Cuando se debe **verificar la interacción** (qué se llamó, cuántas veces, con qué argumentos)	`mock(ProfileJpaRepository.class)` en ms-iam; `HttpTestingController` en el frontend
Dummy	Cuando el parámetro no se usa en el escenario (valores mínimos `Guid.NewGuid()`, nulos tolerados)	Argumentos de conveniencia en las llamadas de `ExecuteAsync`

Regla práctica: **preferir fakes para repositorios** (estado realista y reutilizable), **stubs para servicios externos sin estado**, y **mocks solo cuando la verificación de interacción aporte valor de regresión**. Evitar mocks innecesarios para reducir acoplamiento de las pruebas al detalle de implementación.

8. Datos de prueba

8.1 Clasificación de datos por dominio (PE y AVL)

Tabla 27

Clasificación de datos por dominio (PE y AVL)

Dominio	Datos válidos	Datos inválidos	Valores límite a probar
Identificación de persona	Documento numérico no vacío, único	`null`, `""`, espacios, duplicado de otro registro	Longitud mínima/máxima del documento; `0000000000`
Placa de bus	Placa existente en catálogo (p. ej. `ABC123`)	Placa duplicada, en blanco	Placa única primera vez vs. segunda vez
Capacidad de bus	1–99 pasajeros	0, negativa, > máx	`0`, `1`, `99`, `100`
Horario de ruta	Lunes a viernes	Sábado, domingo	`Monday` (límite inferior) y `Friday` (límite superior)
Edad/expiración de licencia	Fecha futura	Fecha pasada, `null`	Hoy (vigente) vs. ayer (vencida)
Código OTP	6 dígitos en `[000000, 999999]`	5 o 7 dígitos, letras, vacío	`000000`, `999999`
Expiración OTP/token	Dentro del TTL	Después del TTL	Instant TTL − 1 min, TTL exacto, TTL + 1 min
Contraseña	Longitud entre mín. y máx. de la política, cumple requisitos	Corta, sin requisitos	Longitud `PwdMinLength − 1`, `PwdMinLength`, `PwdMaxLength`, `PwdMaxLength + 1`
Búsquedas (nombre/correo/cédula/teléfono)	Subcadena presente, mayúsculas, acentos	Cadena ausente, vacía, `null`	Cadena de 1 carácter, cadena completa, sin coincidencias
Query frontend	Parámetros válidos (`search`, `page`, `pageSize`)	Parámetros nulos	Sin resultados → lista vacía; error 404/500 sin cuerpo

8.2 Fixtures, builders y factories

- Los fakes del proyecto ya actúan como *factories* de datos: `InMemory*Repository` puede precargarse con entidades de ejemplo al crear la instancia.
- En `ms-route` y `sg-ms-forgot-information` se usan proveedores de tiempo fijos (`FixedTimeProvider`, `ManualTimeProvider`) como *fixture temporal*.

- En el frontend, los specs construyen **tokens JWT de prueba** mediante una función auxiliar local (`jwt(claims)` que codifica header/claims en base64) y respuestas de `flush` tipadas para cada servicio.
- Para nuevas pruebas se recomienda centralizar **builders** (p. ej. `TestBus()`, `TestStudent()`, `TestOtpRequest()`) en la carpeta `Fakes/` de cada proyecto de pruebas, reutilizando las entidades y DTOs reales del dominio.

8.3 Datos en memoria y archivos de apoyo

- **Datos en memoria**: colecciones `List<T>`/`Dictionary` dentro de los fakes (sin E/S), ideales para el aislamiento exigido.
- **Archivos de apoyo**: no se requieren para las pruebas unitarias; si fuera necesario (expresiones o plantillas muy extensas), se ubicarían como *embedded resources* del proyecto de pruebas y se leerían sin tocar el sistema de archivos real (evitar `File.ReadAllText` en el SUT).

9. Ejecución de las pruebas

9.1 Ejecución local (sin contenedor)

Microservicios .NET (por servicio):

```
cd <servicio>
dotnet restore
dotnet test tests/<servicio>.Tests/<servicio>.Tests.csproj \
  --configuration Release \
  --collect:"XPlat Code Coverage" \   # o "--collect:Code Coverage" en proyectos sin coverlet
  --logger "trx;LogFileName=resultados.trx"
```

IAM (Java):

```
cd ms-iam
./mvnw test          # JUnit 5 vía spring-boot-starter-*-test
```

Frontend:

```
npm ci
npx ng test --watch=false   # Vitest vía @ngx-env/builder:unit-test (jsdom)
```

Tiempos y reportes: con `--collect` se generan XML de cobertura por proyecto y con `--logger trx` se obtienen reportes de resultados; en Vitest el reporte por defecto se muestra en consola y puede exportarse a JUnit/HTML añadiendo los reporters correspondientes.

Nota (NUnit): `dotnet test` ejecuta las pruebas NUnit a través de `NUnit3TestAdapter`; el proyecto de pruebas referencia el paquete `NUnit` además del `Microsoft.NET.Test.Sdk`.

9.2 Ejecución contenerizada (Docker dedicado solo para pruebas)

El objetivo es **empaquetar una copia del proyecto** en un contenedor desechable que ejecute únicamente las pruebas unitarias, sin exponer puertos ni requerir la infraestructura real (SQL Server, Kafka, SMTP).

Tabla 28

9.2.1 Imagen base por stack

Stack	Imagen base (SDK, no runtime)	Justificación
.NET 10	`mcr.microsoft.com/dotnet/sdk:10.0`	Compilar y ejecutar las pruebas .NET (NUnit) sin runtime adicional
Java 21 + Maven	`maven:3.9-eclipse-temurin-21`	Compilar y ejecutar JUnit/Mockito
Angular/NPM	`node:22-alpine`	`npm ci` + Vitest en jsdom

9.2.2 Ejemplo: Dockerfile para un microservicio .NET

```
# Dockerfile.test — solo para ejecutar pruebas unitarias
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS test
WORKDIR /app

# 1. Restaurar dependencias (capa cacheable)
COPY ms-fleet/src/ms-fleet.Api/ms-fleet.Api.csproj ms-fleet/src/ms-fleet.Api/
COPY ms-fleet/tests/ms-fleet.Tests/ms-fleet.Tests.csproj ms-fleet/tests/ms-
fleet.Tests/
RUN dotnet restore ms-fleet/tests/ms-fleet.Tests/ms-fleet.Tests.csproj

# 2. Copiar el código fuente del servicio y de las pruebas
COPY ms-fleet/ ms-fleet/
```

```
# 3. Ejecutar las pruebas con cobertura y reporte
CMD ["dotnet", "test", "ms-fleet/tests/ms-fleet.Tests/ms-fleet.Tests.csproj", \
     "--configuration", "Release", \
     "--collect:XPlat Code Coverage", \
     "--logger", "trx;LogFileName=resultados.trx"]
```

9.2.3 Ejemplo: docker-compose para el conjunto de pruebas

```
# docker-compose.test.yml — orquestación de contenedores de prueba (sin servicios
reales)
services:
  test-fleet:
    build: { context: ., dockerfile: Dockerfile.test }
    volumes:
      - ./reportes/fleet:/app/TestResults   # exportar TRX + cobertura
    networks: [test]
  test-route:
    build: { context: ms-route, dockerfile: Dockerfile.test }
    volumes:
      - ./reportes/route:/app/TestResults
    networks: [test]
  test-iam:
    image: maven:3.9-eclipse-temurin-21
    working_dir: /app
    volumes:
      - ./ms-iam:/app
      - ./reportes/iam:/app/target/surefire-reports
    command: ./mvnw test
    networks: [test]
  test-web:
    image: node:22-alpine
    working_dir: /app
    volumes:
      - ./dvlp-web:/app
    command: sh -c "npm ci && npx ng test --watch=false"
    networks: [test]

networks:
  test: {}
```

Comandos de orquestación:

```
# Todo el conjunto (cada contenedor levanta, corre y muere; sin puertos expuestos)
docker compose -f docker-compose.test.yml up --build --abort-on-container-exit

# Un solo servicio
docker compose -f docker-compose.test.yml run --rm test-iam

# Limpieza posterior
docker compose -f docker-compose.test.yml down
```

9.2.4 Notas para el contenedor de pruebas

- No se levantan SQL Server, Kafka, SMTP ni el gateway: las pruebas usan exclusivamente fakes y mocks.

- Los volúmenes montados solo exportan reportes (`trx`, XML de cobertura, `surefire-reports`, HTML de Vitest) para evidenciar los resultados.
- En CI se recomienda un job por servicio para aislar fallos y paralelizar la ejecución.

10. Resultados

10.1 Estado de la ejecución

Al cierre de la presente versión, **las pruebas no se han ejecutado** en un entorno de verificación formal; por tanto, todos los campos de resultado se marcan como **«No ejecutado»**. Los conteos de pruebas **existentes** provienen de una inspección estática del código (inventario), no de una corrida.

Tabla 29

Estado de la ejecución

Módulo	Pruebas existentes (inventario estático)	Ejecutadas	Aprobadas	Fallidas	Omitidas	Tiempo
ms-configuration	16 (NUnit)	No ejecutado	—	—	—	—
ms-exceptional	17 (NUnit)	No ejecutado	—	—	—	—
ms-fleet	25 (NUnit)	No ejecutado	—	—	—	—
ms-route	45 (NUnit)	No ejecutado	—	—	—	—
ms-school-management	15 (NUnit)	No ejecutado	—	—	—	—
ms-user-management	41 (NUnit)	No ejecutado	—	—	—	—
sg-ms-forgot-information	47 (NUnit)	No ejecutado	—	—	—	—
ms-iam	18 (JUnit @Test)	No ejecutado	—	—	—	—
Backend (total)	**224 métodos de prueba**	No ejecutado	—	—	—	—
Frontend (dvlp-web)	64 archivos `*.spec.ts`	No ejecutado	—	—	—	—
ms-notification	0 (no existe proyecto de pruebas)	—	—	—	—	—

10.2 Cómo generar los reportes automáticamente

Tabla 30

Al ejecutar la sección 9 se generan automáticamente

Reporte	Comando	Artefacto
Resultados .NET	`dotnet test --logger "trx;LogFileName=resultados.trx"`	`.trx` (Visual Studio Test Results)
Cobertura .NET	`dotnet test --collect:XPlat Code Coverage` (o `--collect:Code Coverage`)	`coverage.cobertura.xml` / `*.xml`
Resultados Java	`./mvnw test`	`target/surefire-reports/*.txt` y `*.xml`
Cobertura Java (recomendado)	`./mvnw test` previa incorporación de JaCoCo (ver sección 12)	`target/site/jacoco/*` (HTML/XML)
Frontend	`npx ng test --watch=false`	Consola + reportes JUnit/HTML si se agregan reporters de Vitest

Las evidencias (reportes y capturas) se anexarán por el responsable en la sección 16 como se acordó en el proyecto.

12. Defectos encontrados

Hallazgos de la revisión estática del código (estado de la prueba y su configuración), previos a la ejecución de la batería:

Tabla 31

Defectos encontrados

ID	Defecto	Módulo afectado	Causa probable	Corrección aplicada / recomendada	Estado	Ticket
DEF-01	**No existe proyecto de pruebas unitarias**	`ms-notification`	El servicio se creó sin batería de pruebas (los demás servicios tienen `tests/{servicio}.Tests`)	Crear `ms-notification.Tests` y cubrir `ProcessScanService`, `RegisterDeviceService`, `GetGuardianNotificationsService` y `StudentScannedConsumer` (casos NOT-01..NOT-04)	Por corregir	— (crear en Jira/GitHub Issues)
DEF-02	**Cobertura de pruebas insuficiente en ms-iam**; sin pruebas para login, tokens y filtros JWT	`ms-iam`	Solo 6 clases probadas (18 `@Test`); `LoginService`, `RefreshTokenService`, `LogoutService`, `CreateProfileService`, `JwtTokenProvider`, `JwtAuthenticationFilter`, `GetTermsStatusService`, `RecordTermsAcceptanceService`, `ListTermsAcceptancesService`, `UpdateAdminSchoolService`, `PersonCreatedConsumer` sin pruebas	Implementar los casos IAM-01 a IAM-11 de este documento (PU-023..PU-028)	Por corregir	—
DEF-03	Directiva `using Xunit;` duplicada	`ms-fleet` (`CreateBusServiceTests.cs`, líneas 1–2)	Copia/pegado al crear el archivo	Eliminar la duplicación (corrección trivial)	Por corregir	—

ID	Defecto	Módulo afectado	Causa probable	Corrección aplicada / recomendada	Estado	Ticket
DEF-04	**JaCoCo no configurado** (sin métricas de cobertura Java)	`ms-iam` (`pom.xml`)	El `pom.xml` declara solo `spring-boot-maven-plugin` y `maven-compiler-plugin`	Agregar `org.jacoco:jacoco-maven-plugin` con preparación de `argLine` y reporte de verificación	Por corregir	—
DEF-05	**Cobertura .NET no uniforme**: `coverlet.collector` ausente en 4 proyectos	ms-fleet, ms-route, ms-user-management, ms-forgot-information	Solo 3 de 7 proyectos declararon Coverlet	Añadir `coverlet.collector` a los 4 proyectos o estandarizar el collector integrado del Test SDK en todos	Por corregir	—
DEF-06	**Sin integración continua** (no hay workflows `.github/workflows` en backend ni frontend)	Infraestructura	Las pruebas solo se ejecutan localmente	Crear pipelines de CI con un job por servicio y umbral de cobertura; ejecución contenerizada según sección 9	Por corregir	—
DEF-07	**Smoke tests débiles en páginas del frontend**	Frontend (p. ej. `buses.spec.ts`)	Varias especificaciones de páginas verifican solo `should create`	Ampliar con pruebas de comportamiento (carga de datos, filtros, estados de error) alineadas a FE-06..FE-09	Por corregir	—

13. Mantenimiento de las pruebas

13.1 Cuándo actualizar las pruebas

- **Cambio de requisitos**: toda nueva regla de negocio (o cambio en una existente) exige actualizar la prueba correspondiente **en el mismo cambio** (TDD para funcionalidad nueva).
- **Refactorización**: si el comportamiento no cambia, la batería debe permanecer verde; una refactorización que rompe pruebas indica acoplamiento a detalle de implementación y debe corregirse en el diseño.

- **Cambios de contrato/DTO**: se revisan `PersonProfile`, `BusViewFieldsTests`, mapeos y proyecciones al modificar DTOs.
- **Correcciones de defectos**: cada corrección se acompaña de su prueba de regresión (ver sección 12).

13.2 Pruebas inestables (flaky) u obsoletas

- Toda prueba flaky se aísla y se marca con `[Category("Flaky")]` (NUnit) / `@Tag("flaky")` (JUnit) / `it.skip` temporal (Vitest) y se crea un defecto en el tablero.
- Una prueba con `skip` por más de dos iteraciones se **elimina o reescribe**; no se acumulan skips indefinidos.
- Las pruebas obsoletas (SUT eliminado o contrato cambiado para siempre) se eliminan en el PR que retira la funcionalidad.

13.3 Revisión en code review (calidad de la prueba)

En la revisión de cada PR se verifica que las pruebas nuevas: (1) prueben una sola cosa con nombre descriptivo; (2) no toquen BD/red/archivos; (3) cubran casos límite, negativos y excepciones; (4) sigan el patrón AAA y las convenciones de la sección 4; (5) no sean frágiles frente al orden de ejecución; y (6) mantengan la cobertura objetivo mínima del 80 % en lógica de negocio.

15. Conclusiones y recomendaciones

15.1 Cumplimiento de criterios

- El marco técnico de pruebas está **presente y definido** para 7 de los 8 servicios .NET (**NUnit** + fakes en memoria + proveedores de tiempo controlados) y en el frontend (Vitest + mocks HTTP), con 224 métodos de prueba en backend y 64 especificaciones en el frontend.

- **No se cumple** hoy el criterio global del 80 % de cobertura ni el 100 % de pruebas aprobadas: persisten huecos críticos (`ms-notification` sin pruebas; `ms-iam` con 18 de las unidades críticas sin cubrir; viajes y paradas de `ms-route` sin pruebas) y falta configurar la medición de cobertura en 5 proyectos.
- La ejecución formal queda pendiente (sección 10) y es el siguiente hito para validar los criterios de salida.

15.2 Riesgos y limitaciones

- **Riesgo de regresión en seguridad**: autenticación, renovación de tokens y filtros JWT de `ms-iam` no tienen pruebas hoy (RF-02). Impacto: alto.
- **Riesgo operativo**: `ms-notification` es responsable del control de subida/bajada y de las notificaciones a acudientes (RF-04/RF-05/RF-08) y carece de pruebas. Impacto: alto.
- **Sin CI**: la ausencia de pipelines hace que los cambios puedan romper la batería sin detectarse.
- **Limitación**: los conteos de la sección 10 son un inventario estático; los números reales (aprobadas/fallidas/tiempo) se confirmarán tras la primera ejecución.

15.3 Mejoras sugeridas

1. Crear el proyecto de pruebas de `ms-notification` y completar los casos NOT-01..NOT-04.
2. Completar la batería de `ms-iam` (IAM-01..IAM-11) con Mockito, incluidas las pruebas del ciclo del token.
3. Añadir JaCoCo a `ms-iam` y `coverlet.collector` uniforme en todos los proyectos .NET (cierra DEF-04 y DEF-05).
4. Crear pipelines de CI con un job por servicio, umbral de cobertura del 80 % en `Application`/`Domain` y reportes automáticos (cierra DEF-06).

5. Ampliar las especificaciones *smoke* del frontend con pruebas de comportamiento (cierra DEF-07).
6. Adoptar TDD para toda funcionalidad nueva y usar los builders/fixtures de la sección 8.2.

16. Anexos

Anexo A — Plantilla de caso de prueba unitaria

Tabla 32

Plantilla de caso de prueba unitaria

Campo	Contenido
ID	(consecutivo PU-###)
Nombre	`Metodo_Condicion_Resultado`
Unidad probada	Clase y método del SUT
Requisito relacionado	RF-xx.y
Descripción y objetivo	Qué se verifica y por qué
Precondiciones y configuración	Fixtures/mocks utilizados y su estado inicial
Datos de entrada	Valores concretos (incluir límites)
Resultado esperado	Comportamiento/valor/excepción esperada
Resultado obtenido	— (se llena tras la ejecución)
Estado	No ejecutado / Aprobado / Fallido
Tipo	Positiva / Negativa / Valor límite / Excepción

Anexo B — Reportes generados por herramientas

Responsable: el autor de las evidencias (según lo acordado). Se adjuntarán los artefactos producidos por los comandos de la sección 9.2: TRX y XML de cobertura de cada servicio .NET, `surefire-reports` y reporte JaCoCo de `ms-iam`, y reporte de Vitest del frontend, junto con el resumen de la tabla de la sección 10 actualizada.

Anexo C — Ejemplo de prueba bien escrita (código real del proyecto)

Backend (.NET) — `CreateBusServiceTests.cs` (ms-fleet), en sintaxis NUnit:

```
[Test]
public async Task ExecuteAsync_WithDuplicatePlate_ShouldThrow()
```

```
{
    var repo = new InMemoryBusRepository();
    var gpsService = new FakeGpsDeviceService();
    var useCase = new CreateBusService(repo, gpsService);

    await useCase.ExecuteAsync(
        Guid.NewGuid(), DateTime.UtcNow.AddYears(1), Guid.NewGuid(),
        40, "ABC123", 1);

    Assert.That(async () => await useCase.ExecuteAsync(
            Guid.NewGuid(), DateTime.UtcNow.AddYears(1), Guid.NewGuid(),
            40, "ABC123", 1),
        Throws.TypeOf<ArgumentException>());
}
```

Frontend (Vitest) — `auth.service.spec.ts` (dvlp-web):

```
it('guarda profileId/personId/email desde la respuesta de login', () => {
  let done = false;
  service.login('admin@school.com', 'secret').subscribe(() => (done = true));

  httpMock
    .expectOne((req) => req.url.endsWith('/login'))
    .flush({
      accessToken: jwt({ roleId: 1 }),
      profileId: '10',
      personId: '20',
      email: 'admin@school.com',
    });

  expect(done).toBe(true);
});
```

Anexo D — Glosario

Ver las definiciones de las secciones 1.3 y 7.2 (unidad, mock, stub, fake, fixture, cobertura, SUT, AAA, PE, AVL, TDD).

Anexo E — Historial de cambios

Tabla 33

Historial de cambios

Versión	Fecha	Cambio
1.0	2026-10-10	Versión inicial (estrategia, unidades a probar, casos PU-001..PU-032, dependencias, datos, ejecución Docker y hallazgos DEF-01..DEF-07).

Documento de pruebas unitarias — School Guardian · SENA, Tecnólogo ADSO, ficha 3145556 · 10 de octubre de 2026.